Completeness of Relational Operations in LLMs:
Benchmark Construction and Preliminary Results

Daryl Alfaro

October 12, 2026

Abstract

We measure whether large language models return *complete* answers when asked for every item that satisfies a condition. We built a 500-question benchmark covering enumeration and SQL-like relational operations (selection, projection, JOIN, aggregation, nested queries and set operations) over synthetic data whose correct answers are computed by executing SQL. We evaluated five models from three families. Across 465 set-valued questions, mean recall is high (0.957–0.986) and precision higher still, but exact-set accuracy ranges from 97% down to 81%: the dominant error is omission. Exact-set accuracy falls as the number of correct items grows (for one model, from 100% at 5 items to 38% at 100), models that stay complete spend several times more reasoning tokens on larger answer sets, and instructing models to be exhaustive or to check records one by one helps only one of four models. One model fabricates plausible items on long lists (991 made-up items on a single question). Re-running individual failures shows that some are systematic and others are run-to-run variation.

1 How the benchmark works

1.1 Synthetic data with executable ground truth

All data in the in-context and retrieval settings is generated from a fixed random seed: 575 tables containing 110,146 records about fictional employees, products, students, books, customers, orders, projects, ID badges and club rosters. Names, cities, brands, titles and codes are invented, so a model cannot answer from memorised knowledge and must read the records it is given. Every question is generated together with the SQL query that defines it, and its gold answer is the result of running that query in SQLite. No answer is written by hand, and an automated test re-executes every query and checks it against the stored answer.

1.2 Controlling answer size and table size

For in-context enumeration ("list all employees whose department is Legal"), the generator assigns the target value to exactly g rows of an n-row table, with $g \in \{5, 10, 20, 50, 100\}$ and $n \in \{50, 200, 500, 1000\}$ ($g \leq n/2$). The matching rows are placed at random positions, which are stored so that misses can be located. For the other operations, the generator repeatedly samples random queries from templates, executes them, and keeps the first whose result size falls in a target range. Each filtering condition is produced in SQL and in English from a single source, so the two cannot disagree.

1.3 Settings and operations

- **In context** (400 questions): all records are in the prompt, so any omission is the model's.

- **Memory only** (60): closed lists (U.S. states, chemical elements, presidents' surnames) filtered by properties computed in code, e.g. states ending in "a" or elements with atomic number 21–36.
- **Retrieval** (40): BM25 retrieves the top $k \in \{10, 20, 50\}$ records from a 200- or 500-record corpus. Recall is also reported over only the correct records that were retrieved.

Records are rendered as Markdown tables, CSV, JSON Lines or English sentences, rotating across questions. Table 1 lists the slices.

Slice	Questions	Median answer size	Max answer size	Median context (k tokens)
Enumeration (in context)	100	20	100	7.6
Enumeration (memory)	60	6	50	0.0
Enumeration (RAG)	40	10	40	1.1
Selection	60	7	92	4.1
Projection	40	15	39	2.9
JOIN	60	11	55	3.4
Aggregation (groups)	50	19.5	367	2.8
Nested	40	8.5	32	2.7
Set operations	50	31.5	97	0.7

Table 1: Benchmark composition (500 questions). Answer size is the number of correct items (for aggregation, the number of records aggregated).

1.4 Prompting and model access

Each prompt contains the data, the question, and a required JSON output format (`{"answer": [...]}`). Three prompt variants are used: *direct* (answer only), *exhaustive* (an explicit instruction to include every qualifying item) and *rowwise* (check each record, then answer). We evaluated five models: Claude Haiku 5.5 through the Claude Code CLI (2.1.296 (Claude Code)) with no tools and a fixed system prompt; DeepSeek V4 Pro and V4 Flash through the OpenCode Go API; and MiMo-V2.6 Pro and Flash through OpenRouter. API calls used temperature 0 and a 32,768-token output limit; the Claude CLI does not expose a temperature setting. All models used their default reasoning ("thinking") behaviour.

1.5 Scoring

The scorer extracts the last JSON answer from the response and normalises case, accents, whitespace and number formatting. For set answers it computes precision, recall, F1 and exact-set match, and classifies every error as *missing* (a correct item omitted), *wrong* (an item present in the data that does not satisfy the question), *made-up* (an item not present in the data) or *duplicate*. Scalar aggregates (COUNT, SUM, MIN, MAX, AVG) are scored by exact match and signed error, so undercounting can be distinguished from overcounting. Responses cut off by the output limit are reported separately.

1.6 Experiments and reproducibility

1. **E1:** all 500 questions, direct prompt, every model.
2. **E2:** a stratified 100-question subset (every fifth question) with the exhaustive and rowwise prompts.
3. **E3:** the same subset, direct prompt, sampled a second time to measure run-to-run variation.

Every call is logged with the full response, the served model identifier, token usage, latency, timestamps, request parameters and a hash of the exact prompt. Each run writes a manifest with the code commit (the first run used `09d66f2`), a hash of the benchmark files (`b2bc2242be80...`), the question identifiers and all software versions. Code and data: https://github.com/daryl-888/llm-relational-completeness.

2 Results

All numbers are from experiment E1 (direct prompt, first sample) unless stated otherwise. Five models completed all 500 questions. MiMo-V2.6 was run on OpenRouter after OpenCode Go usage caps stalled it; its partial OpenCode Go runs, and a Haiku 4.5 run stopped after 90 questions, are excluded from the tables. Responses were re-scored from the saved text with the final version of the scorer.

2.1 Overall: high recall, but incomplete answers are common

Table 2 summarises the 465 set-valued questions. Recall is between 0.957 and 0.986 and precision between 0.975 and 0.998, yet only 81–97% of answers are exactly the correct set. Errors are overwhelmingly *missing* items: apart from MiMo-V2.6 Flash, the models together returned 32 wrong items and no made-up ones. (Most missing items in Table 2 come from the retrieval slice, where they are caused by retrieval; on the 365 in-context set questions the models missed 16, 43, 75, 104 and 492 items, in the order of Table 2.) MiMo-V2.6 Flash is the exception: it returned 1,032 items that do not exist in the data, 991 of them (with 245 duplicates) on a single 100-item question. Twenty of its made-up items come from one answer that named the right products but dropped their model codes ("Dornor" for "Dornor M-670"); because base names repeat across products, these do not identify the items and are scored as wrong. Three answers ran to the output limit (in one, MiMo-V2.6 Pro's reasoning repeated the word "Office" until the limit) and one was refused by a content filter (Haiku 5.5, listing the elements with atomic numbers 55–86).

2.2 By operation

Tables 3 and 4 break results down by slice. Set operations are solved almost perfectly by every model. Enumeration in context, JOIN, projection and grouped aggregation show the largest gaps between recall and exact accuracy. In the memory-only setting, the DeepSeek models answered every question exactly, while the other three missed members of short closed lists (Haiku 5.5, for example, listed Kentucky but not New Jersey as a U.S. state ending in "y"). Retrieval-setting recall (0.87) is identical for all models and equals the retrieval step's recall: every model returned every correct item that was retrieved (Table 6), so with this BM25 configuration the retrieval slice measures the retriever, not the model.

2.3 Answer-set size

For in-context enumeration, Table 5 and Figure 2 show exact-set accuracy falling as the number of correct items grows. At 5 items four of five models are exact on at least 96% of questions; at 50 items Haiku 5.5 is exact on 50%, MiMo-V2.6 Pro on 62% and MiMo-V2.6 Flash on 31%. The two DeepSeek models stay at 88–100%. Recall falls more gently, because the typical failure drops a few items from a long list. Recall also falls with table size for MiMo-V2.6 Flash (from 1.00 at 50 rows to 0.83 at 1,000 rows).

2.4 The cost of completeness

Reasoning effort grows with answer size for the models that stay complete (Figure 1): the median reasoning length rises from roughly 1,000–2,000 tokens at 5 items to roughly 9,000 at 100 items for the DeepSeek models, and from 170 to 6,400 tokens for Haiku 5.5. MiMo-V2.6 Flash frequently answers after only 10–15 reasoning tokens on the larger questions, and those answers account for most of its omissions and fabrications. This is a correlation across questions, not a controlled manipulation of reasoning effort.

2.5 Position in the table

Figure 3 shows the miss rate of correct rows by their position in the table. For MiMo-V2.6 Flash the miss rate rises steadily with position, from about 2% for rows near the top of the table to 11–13% near the bottom. The other models miss too few rows for a clear pattern; Haiku 5.5's misses are concentrated in the first half. There is no U-shaped "lost in the middle" profile.

2.6 Aggregation

On the 35 scalar aggregation questions (Table 7), both DeepSeek models answered every question exactly. Every error by the other three models was an undercount, except two MiMo-V2.6 Flash overcounts, which is consistent with items being dropped before counting.

2.7 Prompting

On the 100-question subset (Table 8), the alternative prompts made no difference for DeepSeek V4 Flash or Haiku 5.5 (recall within 0.003 of the direct prompt). MiMo-V2.6 Pro improved, from 0.963 to 0.972 with the exhaustive instruction and to 0.985 when asked to check each record first. MiMo-V2.6 Flash got slightly worse with the exhaustive instruction (0.960 to 0.937). DeepSeek V4 Pro's prompt-variant runs had not finished under its provider's usage caps and are not reported.

2.8 Run-to-run variation and reproducible failures

Two independent samples of the same 100 prompts gave the same exact/inexact verdict on 87–100% of questions (Table 9). To test whether individual failures are systematic, 13 failures were re-run three more times on the same model (Table 10). Some reproduce every time: Haiku 5.5 omitted the same U.S. states from memory in all three re-runs, and DeepSeek V4 Pro and MiMo-V2.6 Flash both dropped items from the same 50-item list in every re-run. When MiMo-V2.6 Pro failed on that list it omitted the same three employees each time. Other failures, including Haiku 5.5's misses on selection, JOIN, projection and COUNT questions, did not recur. For the stronger models, incompleteness is therefore best described as a rate that rises with answer-set size rather than as a fixed set of questions that always fail.

2.9 Usage

Table 11 lists token usage and cost. Reasoning accounts for 77–97% of output tokens.

Model	Questions	Recall	Precision	Exact	Missing	Near-miss	Wrong	Made-up	Dupl.	Trunc.	Refused
DeepSeek V4 Pro	465	0.986	0.998	0.963	196	0	1	0	0	1	0
DeepSeek V4 Flash	465	0.985	0.997	0.966	223	1	3	0	0	1	0
Haiku 5.5	465	0.978	0.997	0.892	298	0	7	0	0	0	1
MiMo-V2.6 Pro	465	0.973	0.991	0.880	297	0	21	0	0	1	0
MiMo-V2.6 Flash	465	0.957	0.975	0.813	687	7	314	1032	316	0	0

Table 2: E1, direct prompt: mean over set-valued questions (all slices except scalar aggregation). Error columns are total item counts.

Slice	DeepSeek V4 Pro	DeepSeek V4 Flash	Haiku 5.5	MiMo-V2.6 Pro	MiMo-V2.6 Flash
Enumeration (in context)	1.00	1.00	0.99	0.97	0.94
Enumeration (memory)	1.00	1.00	0.95	0.95	0.96
Enumeration (RAG)	0.87	0.87	0.87	0.87	0.87
Selection	1.00	1.00	1.00	0.99	0.97
Projection	1.00	1.00	0.99	1.00	0.99
JOIN	1.00	0.98	1.00	0.99	0.98
Aggregation (groups)	0.93	0.99	0.99	0.98	0.83
Nested	1.00	1.00	0.99	1.00	0.99
Set operations	1.00	1.00	1.00	1.00	1.00

Table 3: E1: mean recall by slice.

Slice	DeepSeek V4 Pro	DeepSeek V4 Flash	Haiku 5.5	MiMo-V2.6 Pro	MiMo-V2.6 Flash
Enumeration (in context)	0.96	0.98	0.83	0.84	0.66
Enumeration (memory)	1.00	1.00	0.87	0.83	0.83
Enumeration (RAG)	0.78	0.78	0.78	0.78	0.78
Selection	0.97	0.98	0.95	0.93	0.88
Projection	1.00	0.95	0.90	0.90	0.82
JOIN	1.00	0.98	0.93	0.88	0.82
Aggregation (groups)	0.93	0.93	0.93	0.87	0.67
Nested	0.97	1.00	0.90	0.93	0.90
Set operations	1.00	1.00	1.00	0.98	1.00

Table 4: E1: fraction of questions answered with exactly the correct set.

Correct items	DeepSeek V4 Pro	DeepSeek V4 Flash	Haiku 5.5	MiMo-V2.6 Pro	MiMo-V2.6 Flash
5	1.00	0.96	1.00	0.91	1.00
10	1.00	1.00	1.00	0.96	0.87
20	1.00	1.00	0.91	0.77	0.55
50	0.88	0.94	0.50	0.62	0.31
100	0.88	1.00	0.56	0.88	0.38

Table 5: E1, in-context enumeration: fraction of questions answered exactly, by number of correct items.

Model	Recall	Recall over retrieved
DeepSeek V4 Pro	0.869	1.000
DeepSeek V4 Flash	0.869	1.000
Haiku 5.5	0.869	1.000
MiMo-V2.6 Pro	0.869	1.000
MiMo-V2.6 Flash	0.869	1.000

Table 6: E1, retrieval setting: recall over all correct items, and over the correct items that retrieval returned.

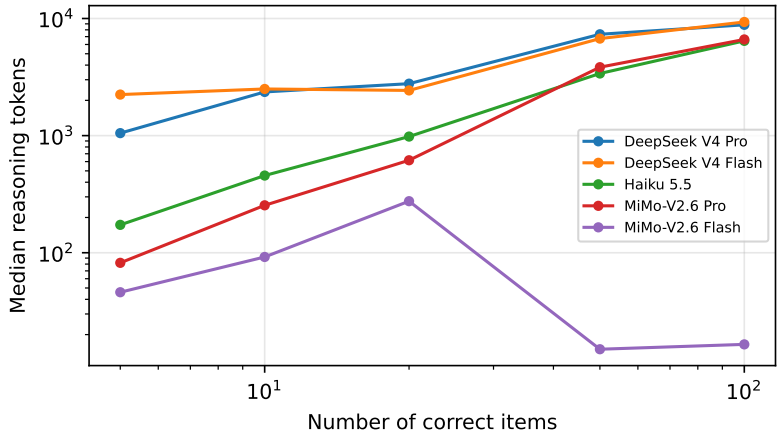


Figure 1: In-context enumeration (E1): median reasoning tokens against the number of correct items.

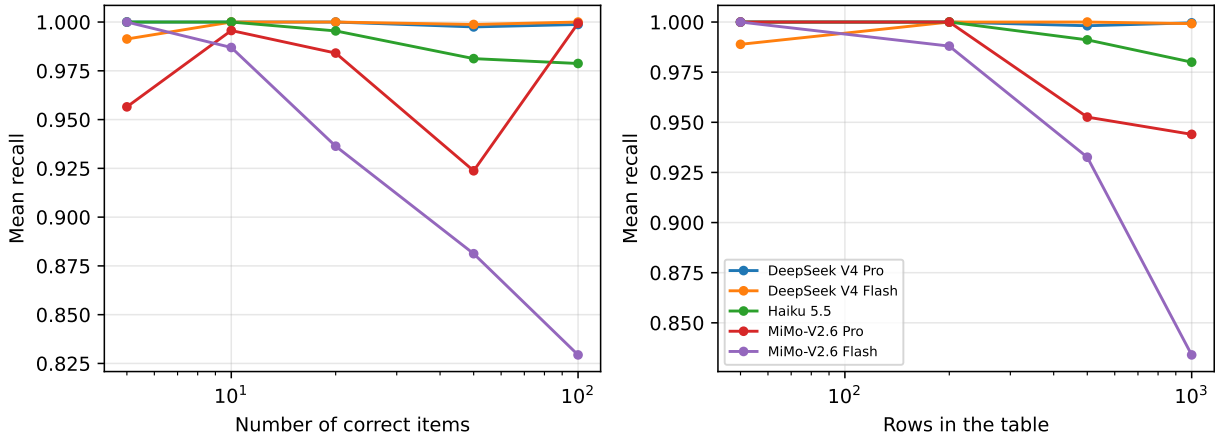


Figure 2: In-context enumeration (E1): mean recall against the number of correct items (left) and table size (right).

Model	Questions	Exact	Undercount	Overcount	Median rel. error
DeepSeek V4 Pro	35	1.00	0.00	0.00	0.00
DeepSeek V4 Flash	35	1.00	0.00	0.00	0.00
Haiku 5.5	35	0.94	0.06	0.00	0.00
MiMo-V2.6 Pro	35	0.91	0.09	0.00	0.00
MiMo-V2.6 Flash	35	0.83	0.11	0.06	0.00

Table 7: E1, scalar aggregation (35 questions): exact answers and direction of error.

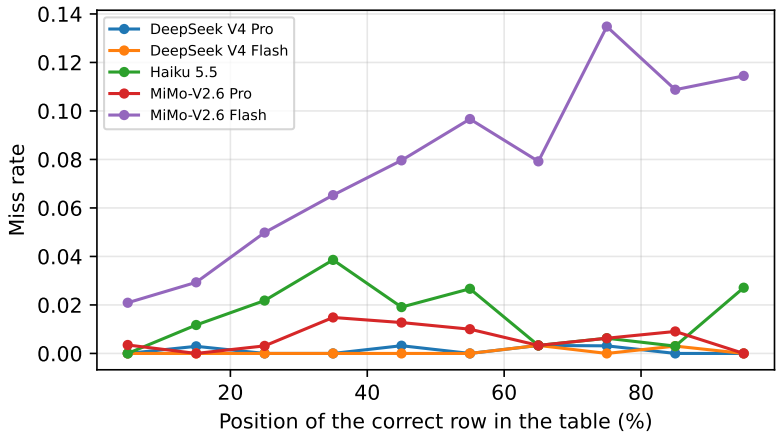


Figure 3: In-context enumeration (E1): fraction of correct rows omitted, by the row's position in the table.

Model	Direct	Exhaustive	Rowwise
DeepSeek V4 Pro	0.989	–	–
DeepSeek V4 Flash	0.989	0.986	0.989
Haiku 5.5	0.983	0.981	0.982
MiMo-V2.6 Pro	0.963	0.972	0.985
MiMo-V2.6 Flash	0.960	0.937	0.958

Table 8: E2: mean recall on the 100-question subset by prompt variant (questions answered under all three prompts).

Model	Questions	Same exact verdict	Mean |recall diff|	Exact (run 1)	Exact (run 2)
Haiku 5.5	100	0.930	0.008	0.920	0.910
DeepSeek V4 Flash	100	1.000	0.000	0.980	0.980
DeepSeek V4 Pro	94	0.979	0.000	0.957	0.979
MiMo-V2.6 Flash	100	0.870	0.035	0.790	0.820
MiMo-V2.6 Pro	100	0.900	0.027	0.870	0.890

Table 9: E3: agreement between two independent runs of the direct prompt on the subset.

Model	Question	Answers	Records	Main-run outcome	Reproduced
Haiku 5.5	`enum-par-013`	2	50	recall 0.50, 1 missing	2/3
Haiku 5.5	`enum-par-021`	11	50	recall 0.73, 3 missing	3/3
Haiku 5.5	`sel-029`	13	400	recall 0.85, 2 missing	0/3
Haiku 5.5	`join-004`	55	100	recall 0.93, 4 missing	0/3
Haiku 5.5	`nest-022`	13	200	recall 0.92, 1 missing	2/3
Haiku 5.5	`proj-031`	37	300	recall 0.89, 4 missing	0/3
Haiku 5.5	`agg-015`	12	400	answered 11 (gold 12)	0/3
Haiku 5.5	`enum-ctx-029`	50	500	recall 0.96, 2 missing	0/3
DeepSeek V4 Pro	`enum-ctx-029`	50	500	recall 0.98, 1 missing	3/3
MiMo-V2.6 Flash	`enum-ctx-029`	50	500	recall 0.94, 3 missing, 3 wrong	2/3*
MiMo-V2.6 Pro	`enum-ctx-029`	50	500	recall 0.92, 4 missing	1/3
MiMo-V2.6 Flash	`enum-ctx-035`	100	1000	recall 0.52, 46 missing, 2 near-miss, 19 wrong, 371 made-up, 34 duplicate	2/3
DeepSeek V4 Flash	`enum-ctx-018`	5	50	recall 0.80, 1 near-miss	0/3

Table 10: Selected failures, each re-run three times on the same model. "Reproduced" counts re-runs that were again not exactly correct (an asterisk marks a re-run with no parseable answer, not counted). Models first run on OpenCode Go were re-run through OpenRouter.

Model	Calls	Input (M)	Output (M)	of which reasoning (M)	Cost (USD)	Median latency (s)
DeepSeek V4 Pro	1021	3.83	2.21	2.14	6.87	23.43
DeepSeek V4 Flash	1307	5.08	3.02	2.81	2.52	12.48
Haiku 5.5	893	8.04	1.88	1.64	2.49	7.70
MiMo-V2.6 Pro	800	5.77	2.00	1.78	3.29	16.65
MiMo-V2.6 Flash	800	5.78	1.72	1.32	1.25	13.10

Table 11: Usage across all experiments (4,821 calls, including retries). Cost is at provider list prices; the Claude models were accessed through a subscription and OpenCode Go usage drew $9.39 from a subscription allowance.